

Convocatòria d'Administracions Públiques

VIBE CODING EDUCATIU AMB INTEL·LIGÈNCIA ARTIFICIAL I LA SEUA PUBLICACIÓ

2025-2026

Del 18 juny al 16 de juliol

A distància

32 Hores



GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Universitats

CFR | CEFIRE
INTEL·LIGÈNCIA ARTIFICIAL I PENSAMENT COMPUTACIONAL

Índice

El Flujo de Trabajo Vibe Coding 2
Los 5 Pasos del Flujo . . . 2
Cómo usar la IA para planificar la implementación . . . 2
Plantilla reutilizable: plan de implementación . . . 3
Ejemplo Aplicado: Cuestionario de Capitales . . . 4
Checklist del Vibecoder . . . 4
* Lo esencial en 1 página . . . 4
Cómo usar este bloque por niveles . . . 4
Comparativa: IA vs programación (general) y HTML5 (en particular) . . . 5
Cómo pedir ayuda a la IA (prompts) . . . 5
Iterar el prompt es parte del proceso . . . 5
IA para construir y IA para planificar . . . 6
Sesgos, errores acumulados y temas delicados . . . 6
Instrucciones y agentes (reglas persistentes) . . . 7
Memoria y contexto del proyecto . . . 7
Cómo pedir a la IA un plan de implementación . . . 8
Seis pasos para planificar el proyecto con la IA . . . 9
Cómo detectar una mala respuesta de la IA . . . 10
Revisión crítica antes de aceptar un cambio . . . 10
Prompts recomendados por nivel . . . 10
Anatomía de una Web App Educativa . . . 11
Catálogo de Prompts PRO a editar y personalizar . . . 11
Guía de Publicación en Aules . . . 12
Consejos Finales de IA . . . 13
Créditos . . . 13
Uso de la consola en navegadores (explicación amplia) . . . 13
IA desde la terminal (CLI): Codex y Claude Code . . . 14
Descripción . . . 17
Propuesta de proyecto . . . 17
Proyectos alternativos equivalentes . . . 17
Ruta por niveles . . . 17
Pasos guiados (Módulo 1 - Versión Mínima) . . . 17
GitHub Pages: paso a paso . . . 19
(Opcional avanzado) ¿Qué es una PWA y qué ventajas ofrece en el aula? . . . 19
Problemas comunes . . . 20
Entregable . . . 20
Cómo usar este recurso en una sesión de clase . . . 20
Variantes por materia . . . 20
Guía mínima de uso . . . 20
Módulo 2: Añadir CSS y JavaScript . . . 21
Módulo 2: Añadir CSS y JavaScript . . . 23

El Flujo de Trabajo Vibe Coding

El `vibe coding` se basa en un ciclo de trabajo simple, rápido e iterativo. En lugar de grandes planes, nos enfocamos en dar pequeños pasos que nos acerquen a nuestro objetivo. Este es el flujo que seguiremos en el curso.

Los 5 Pasos del Flujo

En este bloque, el nivel básico puede seguir el mismo flujo sin necesidad de usar Git desde el primer día. En los niveles medio y avanzado, ese mismo flujo se complementa con `commit` y `push`.

1. Definir un Objetivo Educativo Concreto Todo empieza con una necesidad del aula. ¿Qué problema queremos resolver? ¿Qué proceso queremos facilitar? La clave es ser específico. Un buen objetivo no es "hacer una web de historia", sino "crear un test de 10 preguntas sobre la Revolución Francesa". Cuanto más acotado sea el objetivo, más fácil será empezar y terminar.

2. Dividir en Tareas Pequeñas y Asumibles Una vez tenemos el objetivo, lo rompemos en las partes más pequeñas posibles. Si queremos un cuestionario, las tareas podrían ser: - Crear la estructura HTML. - Añadir una sola pregunta. - Añadir los botones de respuesta. - Programar la lógica para corregir. - Darle estilos con CSS. Cada una de estas tareas puede convertirse en un `commit` en nuestro repositorio de Git.

3. Construir la Versión Mínima Elegimos la tarea más simple que aporte valor y la construimos. Nuestro lema es: "la versión más tonta que pueda funcionar". Para el cuestionario, sería simplemente mostrar una pregunta y las opciones en la pantalla, sin ninguna interactividad.

- **Nivel básico:** guardar el archivo y comprobarlo en el navegador.
- **Nivel medio y avanzado:** cuando funciona, hacer además un `commit` en Git como punto de guardado.

> **[IA] Cómo pedirle a la IA**
>
> "Acabo de terminar la versión mínima. Haz el commit con un mensaje de una línea que describa exactamente lo que se ha añadido."

4. Probar, Ajustar y Mejorar Con la versión mínima funcionando, la probamos. ¿Se ve bien? ¿Es clara? A partir de ahí, cogemos la siguiente tarea pequeña (ej. añadir interactividad) y la implementamos.

- **Nivel básico:** repetir el ciclo `cambiar -> guardar -> probar`.
- **Nivel medio y avanzado:** cada vez que completamos una mejora y comprobamos que todo sigue funcionando, hacer además un `commit`.

> **[IA] Cómo pedirle a la IA**
>
> "He añadido [describe el cambio] y funciona. Haz el commit con un mensaje de una línea que resuma el cambio."

Este ciclo de `construir -> probar -> mejorar` es el corazón del vibe coding.

5. Documentar lo Esencial y Publicar La documentación no es un paso final, sino algo que ocurre en paralelo.

- **Nivel básico:** basta con dejar el recurso entendible y entregable.
- **Nivel medio:** se espera añadir un `README.md` con objetivo, instrucciones y créditos.
- **Nivel avanzado:** los mensajes de `commit`, el `README.md` y la publicación con `git push` ayudan a dejar una versión trazable, segura y compartible.

> **[IA] Cómo pedirle a la IA**
>
> "Crea el archivo README.md con objetivo del recurso, instrucciones de uso y nivel educativo al que va dirigido. Después haz commit y push a GitHub."

Cómo usar la IA para planificar la implementación

Una práctica clave del curso es pedir a la IA un plan antes de pedir código. Así evitamos soluciones demasiado grandes, difíciles de revisar o mal adaptadas al nivel del proyecto.

Secuencia recomendada

1. Explica el objetivo educativo.

2. Pide a la IA que lo divida en etapas.
3. Pide tareas concretas para cada etapa.
4. Pide una comprobación simple al final de cada etapa.
5. Ejecuta solo una etapa cada vez.

Ejemplo de etapas

- **Etapa 1:** estructura base del recurso.
- **Etapa 2:** mejora visual mínima.
- **Etapa 3:** interacción básica.
- **Etapa 4:** refinado, documentación y publicación.

Prompt útil

```
Quiero construir [recurso].
Antes de darme código, divide la implementación en etapas.
Para cada etapa indica:
- objetivo
- tareas pequeñas
- archivos implicados
- comprobación final
```

Ventaja del método

La IA deja de actuar como generador masivo de código y pasa a funcionar como apoyo para organizar, priorizar y validar el trabajo.

Si quieres una versión más formal de este modo de trabajo (especificación, criterios de aceptación y verificación), revisa el apéndice Metodología SDD.

Plantilla reutilizable: plan de implementación

> **Plantilla de aula** Usa este bloque como modelo para pedir a la IA un plan antes de construir el recurso.

Prompt modelo

```
Quiero crear un cuestionario sobre fracciones para 1 ESO.
La primera versión debe ser sencilla y clara.
Antes de darme código, divide la implementación en etapas.
Para cada etapa indica:
- objetivo
- tareas pequeñas
- archivos implicados
- comprobación final
```

Respuesta modelo esperable

Etapa 1: estructura base - **Objetivo:** tener una primera página visible con contenido educativo. - **Tareas:** crear `index.html`, añadir título, añadir una breve explicación, añadir una pregunta con varias opciones. - **Archivos implicados:** `index.html` - **Comprobación final:** al abrir el archivo en el navegador se ve el título, el texto y la pregunta.

Etapa 2: mejora visual mínima - **Objetivo:** hacer que el contenido se lea mejor. - **Tareas:** cambiar fuente, colores, márgenes y tamaño del título. - **Archivos implicados:** `index.html` o `style.css` - **Comprobación final:** la página resulta más clara y ordenada que en la etapa 1.

Etapa 3: interacción básica - **Objetivo:** conseguir que el recurso responda a una acción del usuario. - **Tareas:** añadir botón de corregir, mostrar mensaje de acierto o error, comprobar que el botón funciona. - **Archivos implicados:** `index.html` y, si se separa, `script.js` - **Comprobación final:** al pulsar el botón aparece un mensaje visible en la página.

Etapa 4: mejora funcional - **Objetivo:** hacer el cuestionario más útil. - **Tareas:** añadir una segunda pregunta, mejorar el feedback, revisar textos y errores. - **Archivos implicados:** `index.html`, `script.js` - **Comprobación final:** el cuestionario permite completar más de una pregunta sin fallos visibles.

Etapa 5: cierre y entrega - **Objetivo:** dejar el recurso preparado para compartir o entregar. - **Tareas:**

añadir instrucciones mínimas, revisar ortografía, ordenar archivos, decidir si se publica. - **Archivos implicados:** `index.html`, `README.md` si existe - **Comprobación final:** otra persona puede abrir el recurso, entenderlo y usarlo.

Cómo usar esta plantilla en clase

- Primero se pide solo el plan.
- Después se trabaja únicamente la etapa 1.
- Cuando la etapa 1 funciona, se pide a la IA ayuda para la etapa 2.
- Así sucesivamente, sin mezclar varias etapas en la misma petición.

Ejemplo Aplicado: Cuestionario de Capitales

- **Objetivo:** Crear un cuestionario tipo test sobre las capitales de Europa.
- **Versión 1:** Un `index.html` con una pregunta (`<p>`) y una lista de respuestas (`<ul>`).
- **Versión 2:** Cambiar la lista por `radio buttons` dentro de un `<form>`.
- **Versión 3:** Añadir un `script.js` que muestra un `alert` con el resultado.
- **Versión 4:** Añadir un `style.css` para mejorar la apariencia.

En nivel medio o avanzado, cada una de estas versiones puede corresponder además a un `commit`.

Checklist del Vibecoder

☐ ¿Mi objetivo de hoy es pequeño y claro?
☐ ¿He guardado o registrado un cambio funcional?
☐ ¿He probado mi cambio en el navegador?
☐ ¿Entendería mi "yo" del futuro el código y los mensajes de `commit` que he escrito hoy? # IA, prompts y depuración

* Lo esencial en 1 página

Este bloque es largo porque es **material de referencia**: no hace falta leerlo entero de una vez. Para empezar basta con estos seis puntos; el resto consúltalo cuando lo necesites.

1. **Estructura cada prompt:** *contexto* (qué haces y en qué archivo) + *objetivo* (qué quieres) + *restricciones* (tecnología, público, longitud) + *ejemplo* si hace falta.
2. **Pide cambios pequeños y comprobables**, no todo de golpe. Una mejora cada vez.
3. **Verifica siempre** el resultado en el navegador y con la consola (F12). La IA no prueba el código por ti.
4. **Revisa antes de aceptar:** si no entiendes un cambio, pídele que te lo explique en lenguaje claro.
5. **Nunca datos reales del alumnado en un prompt** (ver la *regla de oro* en Buenas prácticas).
6. **Si una respuesta no funciona, itera:** describe el error, pega el mensaje de la consola y pide una solución pequeña.

Dónde encontrar más (referencia): prompts e iteración · planificación con IA · detectar malas respuestas · consola y depuración · configurar Copilot · IA desde la terminal. Todo ello está desarrollado más abajo en este mismo bloque.

Cómo usar este bloque por niveles

Nivel básico

- Pide cambios pequeños.
- Usa HTML simple.
- Pide explicaciones en lenguaje claro.

Nivel medio

- Añade contexto de archivos, estructura y objetivo.
- Pide CSS y JavaScript por separado.
- Usa la consola para depurar.

Nivel avanzado

- Explora prompts complejos, librerías, recursos para Moodle o diseños más ambiciosos.
- Revisa siempre lo generado antes de usarlo en el aula.

Comparativa: IA vs programación (general) y HTML5 (en particular)

IA (general)

- **Fortalezas:** acelera prototipos, sugiere soluciones, explica errores, genera ejemplos y variantes.
- **Limitaciones:** no garantiza exactitud, puede inventar APIs, no ejecuta ni prueba por ti, y a veces prioriza lo "bonito" sobre lo correcto.
- **Implicación técnica:** el código sigue siendo determinista; la IA no entiende el contexto completo si no se lo das.

Programación tradicional (general)

- **Fortalezas:** control total, comportamiento reproducible, trazabilidad y depuración precisa.
- **Limitaciones:** requiere más tiempo inicial y conocimiento técnico para construir desde cero.
- **Implicación técnica:** los resultados dependen de reglas claras, pruebas y verificación humana.

HTML5 con IA (en particular)

- **Lo que suele funcionar bien:** estructura base (`header`, `main`, `section`), formularios simples, plantillas de landing, ejemplos de accesibilidad (`aria-label`, `alt`).
- **Lo que requiere revisión:** semántica real del contenido, enlaces y rutas, nombres de clases, validación de formularios y compatibilidad entre navegadores.
- **Regla técnica:** todo lo generado por IA debe validarse en el navegador y con el inspector (HTML, CSS y consola).

Uso recomendado

- **IA para:** arrancar rápido, generar borradores y documentar.
- **Tu criterio para:** definir requisitos, revisar la semántica HTML5, probar y publicar.

Cómo pedir ayuda a la IA (prompts)

La IA funciona mejor cuando le das contexto y un objetivo claro. Usa esta estructura simple:

1. **Contexto:** qué estás haciendo y en qué archivo o parte estás.
2. **Objetivo:** qué quieres lograr.
3. **Restricciones:** formato, longitud, público o tecnología.
4. **Ejemplo:** una muestra corta si aplica.

Iterar el prompt es parte del proceso

El primer resultado que devuelve la IA casi nunca es exactamente lo que imaginabas. Esto no es un fallo: es el funcionamiento normal del vibe coding.

La mejora real llega modificando pequeños detalles hasta lograr exactamente lo que buscas. Cada vuelta de iteración es una instrucción más clara que das a la IA, y el resultado mejora con cada paso.

Mentalidad correcta: - El primer resultado es un punto de partida, no el destino. - Si algo no funciona o no se ve bien, describe qué cambia y vuelve a pedir. - Iterar no es perder tiempo: es aprender a comunicarse con precisión.

Ejemplos de iteraciones típicas:

```
Los botones son demasiado pequeños. Hazlos más grandes y con más espacio entre ellos.

El texto de las preguntas es difícil de leer. Aumenta el tamaño de fuente y mejora el contraste.

Ahora añade una cuarta pregunta sobre [tema] con cuatro opciones de respuesta.
```

Cuándo parar y replantear: Si después de dos o tres iteraciones el resultado sigue alejándose de lo que quieres, suele ser más eficiente empezar con un prompt nuevo más específico que seguir corrigiendo encima

de uno que no arrancó bien.

IA para construir y IA para planificar

Conviene distinguir dos usos diferentes:

- **IA para construir:** te ayuda a escribir HTML, CSS o JavaScript, corregir errores y proponer mejoras técnicas.
- **IA para planificar:** te ayuda a ordenar una actividad, una secuencia didáctica, una adaptación o una forma de usar el recurso en clase.

Lo más recomendable es empezar pidiendo planificación y solo después pasar a la construcción. Así es más fácil que el recurso final responda a una necesidad educativa real y no solo a una idea técnica llamativa.

Si quieres profundizar en este enfoque sin complicar la ruta base, puedes consultar el apéndice Metodología SDD.

Sesgos, errores acumulados y temas delicados

Sesgos tipicos (y como corregirlos)

- Tienden a simplificar demasiado si no indicas nivel educativo y profundidad.
- A veces "anglican" el formato (titulos en mayusculas, frases calcadas del ingles) si no fijas idioma y estilo.

Regla práctica: pide el contenido 1-2 cursos por encima y luego adáptalo a tu aula.

Riesgo de acumulacion de errores

Si una primera version tiene un fallo (conceptual, legal o tecnico) y sigues iterando encima, el error suele amplificarse.

Regla práctica: trabaja por fases y valida cada fase (navegador, consola, lectura final) antes de seguir.

Rutina recomendada (muy simple): 1. Define un objetivo pequeño ("que el boton muestre feedback"). 2. Pide un plan de 3-5 pasos, no código. 3. Ejecuta solo el paso 1. Prueba en navegador. 4. Si funciona, guarda (y si usas Git, haz commit). Si no, depura antes de seguir. 5. Repite.

Legislacion y normativa: no trabajar "de memoria"

Cuando el contenido depende de normativa, hay alto riesgo de que la IA invente referencias o mezcle comunidades/fechas.

Flujo recomendado: 1. Aporta documentos oficiales (PDF/TXT) en el proyecto o en el chat. 2. Pide citas textuales cortas y resumen con referencias a secciones/paginas. 3. Revisa siempre con criterio humano antes de publicar o compartir.

Checklist de seguridad (antes de aceptar una respuesta): - La IA indica de qué documento sale cada afirmación importante. - Las citas son cortas y verificables (no "resumen sin fuente"). - Si falta un dato, la IA lo marca como "no encontrado" en vez de inventarlo.

Plantilla de prompt (lectura segura de normativa):

```
Tengo estos documentos oficiales (adjuntos o en el proyecto).
Objetivo: extraer solo lo que aparece en los documentos, sin inventar.
Tarea:
1) Lista los puntos clave relacionados con [TEMA].
2) Para cada punto, incluye: (a) resumen en 1-2 frases, (b) cita textual corta, (c) referencia a seccion/pagina.
Reglas:
- Si no esta en los documentos, escribe "NO CONSTA".
- No uses memoria general ni conocimiento externo.
```

Nota sobre formatos: trabajar con texto plano (por ejemplo, extraer a `.txt`) suele ser más estable que trabajar directamente sobre PDF, pero el criterio importante es el mismo: siempre con fuentes verificables.

> Esta sección trata el uso de la IA con documentos normativos. Si lo que buscas es saber qué implica el RGPD para los recursos que publicas (qué datos recoge tu app, cuándo necesitas aviso de privacidad), consulta la sección *Seguridad de datos en las aplicaciones generadas* del

apunte de Buenas prácticas.

Patrón útil: un chat para generar y otro para criticar

Cuando la tarea es importante, separa roles:

- Chat A: genera borrador.
- Chat B: critica duro (busca fallos, incoherencias, riesgos).
- Chat C: adapta a tu público/tiempo/recursos.

Instrucciones y agentes (reglas persistentes)

Cuando trabajas con IA durante varias sesiones, conviene escribir tus reglas una sola vez y reutilizarlas.

Dos formatos habituales: - Instrucciones globales: reglas que quieres que esten siempre activas. - Instrucciones del proyecto: un archivo del repo con criterios del curso y del recurso.

Que tipo de reglas merece la pena fijar

- Idioma y tono: castellano, claro, sin anglicismos innecesarios.
- Rigor: pedir que explique supuestos y que avise si no esta seguro.
- Estilo de cambios: cambios pequeños, con verificación en navegador, sin reescribir todo.
- Rutas y archivos: no inventar ficheros; respetar la estructura del proyecto.

Plantilla corta de instrucciones para un proyecto educativo

Puedes guardarla en un archivo del proyecto (por ejemplo `copilot-instructions.md` o `INSTRUCCIONES_IA.md`):

```
Idioma: responde siempre en castellano.
Objetivo: estoy creando un recurso educativo para usar en el aula.
Estilo: prioriza claridad y simplicidad. Cambios pequenos y verificables.
Restricciones:
- No inventes archivos ni rutas. Si falta contexto, pregunta.
- No cambies todo el proyecto si pido un ajuste pequeño.
- Da siempre un paso de comprobacion (que mirar en el navegador o consola).
Calidad:
- Si hay riesgo de error (conceptos, normativa, seguridad), avisa y pide fuentes.
```

Patron de trabajo recomendado (A/B/C)

- Chat A: genera borrador o plan.
- Chat B: critica y busca fallos.
- Chat C: adapta al nivel real del grupo y al tiempo de aula.

Memoria y contexto del proyecto

Las instrucciones de la sección anterior son las **reglas** (el *cómo*). La **memoria** es algo distinto: es el **estado** del proyecto (el *qué llevas hecho y por qué*). Conviene no confundirlas.

El problema que resuelve la memoria es muy concreto: **cada chat nuevo empieza de cero**. La IA no recuerda lo que decidisteis ayer, ni por qué descartaste una idea, ni en qué punto te quedaste. Si no se lo cuentas otra vez, volverá a proponer cosas ya descartadas o reescribirá lo que ya funcionaba. Esto se nota sobre todo en recursos que construyes a lo largo de **varias sesiones**.

Qué es "memoria" aquí

No es que la IA "recuerde" sola de forma mágica. La memoria, en la práctica, es **información escrita que la IA puede leer al empezar**. Hay dos vías, y se combinan bien:

- **Memoria manual: un archivo de bitácora.** Tú mantienes un archivo de texto en el proyecto (por ejemplo `CONTEXTO.md` o `BITACORA.md`) donde anotas el estado. Al empezar una sesión, le pides a la IA que lo lea (o se lo pegas). Funciona con cualquier IA, hasta con la web de chat más simple.
- **Memoria automática de las herramientas.** Copilot y, sobre todo, los agentes de terminal (Codex, Claude Code) **leen solos** ciertos archivos del proyecto al arrancar (los archivos de instrucciones que viste antes, con nombres por convención como `AGENTS.md`, `CLAUDE.md` o `copilot-instructions.md`). Algunas herramientas tienen además su propia "memoria" donde van guardando datos que les pides

recordar.

> Idea clave: tanto si la lees tú como si la lee la herramienta, **la memoria vive en archivos del proyecto**. Por eso conviene tratarla como parte del proyecto y guardarla con Git.

Qué merece la pena anotar (y qué no)

Anota lo que a un compañero nuevo le harías falta para retomar el trabajo:

- **Objetivo del recurso:** qué es y para qué nivel/asignatura.
- **Decisiones tomadas y por qué:** "usamos un único HTML para poder subirlo a Aules", "sin librerías externas".
- **Estado actual:** qué partes ya funcionan.
- **Próximos pasos:** qué toca hacer ahora.
- **Lo que NO hay que hacer:** ideas ya descartadas, para que no vuelvan a aparecer.

No anotes el código entero (para eso está el propio archivo) ni datos personales del alumnado.

Plantilla de archivo de contexto

Puedes guardar esto como `CONTEXTO.md` en la carpeta del proyecto:

```
# Contexto del proyecto

## Objetivo
Cuestionario de fracciones para 1 ESO, en un único archivo HTML.

## Decisiones
- Un solo index.html (para subirlo fácil a Aules).
- Sin librerías externas; CSS y JS dentro del propio HTML.
- Idioma: castellano, lenguaje claro para el alumnado.

## Estado actual
- Hechas 3 preguntas con botón de corregir y feedback. Funciona en el navegador.

## Próximos pasos
- Añadir una 4ª pregunta.
- Mejorar el contraste de los botones.

## Qué NO hacer
- No añadir Tailwind ni frameworks.
- No separar en varios archivos.
```

Rutina recomendada

1. **Al empezar la sesión:** pide a la IA que lea el archivo de contexto (o pégaselo). En los agentes de terminal lo hará solo si usas el nombre de archivo que esa herramienta espera.
2. **Trabaja** con el método habitual (por etapas, cambios pequeños, comprobando).
3. **Al terminar:** actualiza el archivo de contexto (estado y próximos pasos). Puedes incluso pedírselo a la IA: *"Actualiza CONTEXTO.md con lo que hemos hecho hoy y lo que queda pendiente."*
4. **Guarda con Git** (`commit`). Así la memoria queda versionada junto al recurso.

> **Por niveles.** En **nivel básico**, con recordar el objetivo y los próximos pasos en un par de líneas es suficiente. La bitácora completa y la memoria automática de los agentes son más propias de **nivel medio/avanzado**, cuando el proyecto crece y dura varias sesiones.

Cómo pedir a la IA un plan de implementación

Antes de pedir código, conviene pedir un plan. Esto ayuda a que la IA no mezcle demasiadas cosas a la vez y te permite trabajar por etapas.

Qué debes pedir

Pídele a la IA que te responda con:

1. etapas del proyecto en orden

2. tareas concretas dentro de cada etapa
3. qué archivo tocar en cada tarea
4. cómo comprobar si esa etapa ya funciona

Plantilla de prompt

```
Quiero crear [recurso].
Es para [nivel / asignatura].
La primera versión debe ser sencilla.
No me des código todavía.
Dame un plan de implementación por etapas.
En cada etapa incluye:
- objetivo
- tareas pequeñas
- archivos a tocar
- comprobación final
```

Ejemplo

```
Quiero crear un cuestionario sobre fracciones para 1 ESO.
La primera versión debe ser muy simple.
No me des código todavía.
Dame un plan de implementación por etapas.
En cada etapa incluye objetivo, tareas pequeñas, archivos a tocar y comprobación final.
```

Cómo debe ser una buena respuesta

- **Etapa 1:** estructura base.
- **Tareas:** crear `index.html`, añadir título, añadir una pregunta.
- **Comprobación:** la página se abre y se ve correctamente.
- **Etapa 2:** estilos básicos.
- **Tareas:** cambiar colores, márgenes y tamaño del título.
- **Comprobación:** el contenido es más legible.
- **Etapa 3:** interacción.
- **Tareas:** añadir botón, mostrar mensaje, probar respuesta.
- **Comprobación:** el botón responde y aparece feedback.

Regla práctica

Cuando la IA te dé el plan, trabaja una sola etapa cada vez. Luego puedes pedir:

```
Ahora vamos solo con la etapa 1.
Dame los cambios mínimos para completarla.
```

Seis pasos para planificar el proyecto con la IA

Antes de pedir código, trabaja estos seis elementos para asegurar que el plan tenga sentido educativo:

1. **Objetivo** — Qué recurso quieres construir.
2. **Contexto** — Para qué nivel educativo y con qué finalidad.
3. **Alcance** — Qué debe incluir la primera versión y qué queda fuera.
4. **Plan por etapas** — Pide a la IA que divida el trabajo en fases.
5. **Tareas de cada etapa** — Pide acciones concretas, pequeñas y verificables.
6. **Criterio de comprobación** — Pide cómo saber si cada etapa está bien acabada.

Ejemplo de etapas esperadas

Etapa	Objetivo	Tareas	Comprobación
01	Crear estructura base	Título, texto inicial, una pregunta visible	La página se abre y se ve en el navegador
02	Mejorar aspecto	Colores, espaciado, legibilidad	El contenido se lee bien

Etapa	Objetivo	Tareas	Comprobación
03	Añadir interacción	Botón, mensaje, corrección simple	Al pulsar, aparece feedback

> **Regla metodológica:** No pasar a la siguiente etapa hasta que la anterior funcione. Si la IA reescribe demasiado, mezcla etapas o propone herramientas no pedidas, hay que parar, simplificar el prompt y pedir una versión más corta.

Cómo detectar una mala respuesta de la IA

Estas señales indican que conviene frenar y pedir una versión mejor:

- **Reescribe demasiado:** cambia toda la página cuando solo pediste un ajuste pequeño.
- **Mete librerías sin pedirlas:** añade Tailwind, Chart.js u otras herramientas sin que sean necesarias.
- **Inventa archivos o rutas:** habla de `app.js`, `styles.css` o carpetas que no existen en tu proyecto.
- **Mezcla varias etapas:** intenta resolver estructura, estilos y lógica a la vez.
- **No explica cómo comprobarlo:** da código, pero no dice qué deberías ver al probarlo.

Qué hacer si ocurre

- Pide cambios mínimos sobre un único archivo.
- Recuerda a la IA qué archivos existen de verdad.
- Pide una sola etapa cada vez.
- Exige una comprobación final breve.

Revisión crítica antes de aceptar un cambio

Antes de copiar una respuesta de la IA a tu proyecto, revisa estas preguntas:

- ¿Responde de verdad al objetivo didáctico que habías definido?
- ¿Cambia solo lo que pediste o reescribe más de la cuenta?
- ¿El código es suficientemente simple para que puedas explicarlo después?
- ¿Podrás comprobar en menos de dos minutos si funciona?
- ¿El resultado es usable por tu alumnado o necesita simplificarse?

Prompt de rescate

```
La respuesta anterior cambia demasiadas cosas.
Trabaja solo sobre [archivo real].
No añadas librerías ni archivos nuevos.
Haz solo el cambio mínimo para esta etapa.
Al final dime cómo comprobar si funciona.
```

Prompts recomendados por nivel

Nivel básico

- "Estoy en `index.html`. Quiero un encabezado y una lista de 3 actividades para alumnado de 3 ESO. Solo HTML simple."
- "Tengo este error en la consola: `Uncaught TypeError`. Explícame qué significa y cómo corregirlo en palabras simples."
- "Escribe 3 preguntas tipo test sobre fracciones para 1 ESO, con 4 opciones y la respuesta correcta."
- "Propón mejoras de accesibilidad para un formulario simple. Resume en 5 puntos."

Consejo rápido: si la respuesta es demasiado larga, pide un resumen; si falta algo, pide que lo complete paso a paso.

Nivel medio

- Pide una modificación concreta en `style.css` o `script.js`.
- Pide fixes mínimos, no reescrituras completas.
- Pide siempre que respete tu estructura de archivos.

Nivel avanzado

- Puedes trabajar con prompts largos, requisitos complejos y recursos enriquecidos, pero exigiendo siempre versiones verificables y un resultado mantenible.

Anatomía de una Web App Educativa

Cuando pedimos a una IA que genere un recurso, estamos integrando tres lenguajes fundamentales:

- **HTML:** La estructura y el contenido (el texto, los botones, las tablas).
- **CSS:** El diseño visual y la estética (colores, tipografías, disposición).
- **JavaScript:** La lógica y la interactividad (cálculos, juegos, sonidos).

Recomendación por nivel

- **Básico:** puedes empezar con un solo `index.html`.
- **Medio:** separa HTML, CSS y JavaScript en archivos distintos.
- **Avanzado:** decide si te compensa usar `Single File HTML`, librerías externas o empaquetados específicos.

Concepto clave: Single File HTML – Para que el material sea tan fácil de compartir como un PDF, puedes pedir a la IA que incluya todo el código en un único archivo `.html`. Así, solo tendrás que subir un archivo a plataformas como Aules o Moodle. Es especialmente útil en nivel básico o para recursos rápidos de compartir.

Opcional avanzado: si quieres ir más allá, pide también un paquete SCORM listo para Moodle. Describe qué comportamientos debe reportar (por ejemplo, porcentaje completado o actividad superada) y solicita un `imsmanifest.xml` válido junto a las carpetas `html`, `media` y `scripts` comprimidas en un `.zip`. Así podrás integrar el recurso en un espacio del aula virtual y obtener feedback automático del avance del alumnado.

Catálogo de Prompts PRO a editar y personalizar

Los siguientes prompts quedan como material de **nivel avanzado**. Se conservan para quien quiera explorar proyectos más ambiciosos.

Situación de Aprendizaje LOMLOE Pro

Ideal para generar estructuras curriculares completas y visuales. Prompt: > Actúa como un experto en didáctica y legislación educativa española. Diseña una aplicación web en un único archivo HTML que presente una Situación de Aprendizaje (SdA) completa para el nivel [NIVEL] de la asignatura [ASIGNATURA] sobre "[TEMA]". > La interfaz debe ser moderna, usando Tailwind CSS, con una barra lateral de navegación para movernos entre: 1. Identificación y Justificación (con ODS), 2. Concreción Curricular (Competencias y Criterios), 3. Saberes Básicos, 4. Secuencia de Actividades (Inicio, Desarrollo y Cierre) y 5. Evaluación (incluyendo una rúbrica interactiva en tabla). > El diseño debe incluir tarjetas visuales, iconos de FontAwesome y un botón para exportar el contenido a PDF si es posible. Asegúrate de que los criterios de evaluación estén vinculados directamente con las actividades propuestas.

Unidad de Inglés Profesional (Business English)

Enfocado a la cultura de negocios y práctica comunicativa. Prompt: > Diseña una unidad didáctica interactiva en un único html sobre Networking Profesional en EE. UU. dirigida a estudiantes de inglés de nivel intermedio (B1-B2). La interfaz debe ser limpia y moderna, utilizando una paleta de colores suaves (tonos pastel, azules y naranjas) y una estructura de navegación sencilla con una barra lateral que permita moverse entre cinco secciones: Lectura, Audición, Vocabulario, Gramática y Práctica Oral. > El objetivo principal es que el usuario aprenda no solo el idioma, sino también la cultura de negocios estadounidense. Para el contenido académico, incluye un texto de lectura sobre la etiqueta empresarial en EE. UU., dos secciones de vídeo (una general y otra para personas introvertidas) con cuestionarios de opción múltiple, y una lista de vocabulario clave. La sección de gramática debe explicar de forma sencilla el uso de preguntas indirectas educadas y el Presente Perfecto, acompañando cada explicación con ejercicios prácticos. Finalmente, añade una sección de Speaking con tarjetas de rol.

Laboratorio de Funciones Matemáticas

Simulador visual con gráficos dinámicos. Prompt: > Crea una aplicación educativa interactiva en un único archivo HTML llamada 'Laboratorio de Funciones Matemáticas'. La interfaz debe dividirse en dos columnas: una principal a la izquierda para la visualización y controles, y una lateral a la derecha para la teoría y ejemplos. > En la columna principal, incluye un selector para elegir entre diferentes tipos de funciones (Constante, Lineal, Cuadrática, etc.). Debajo, debe aparecer un gráfico dinámico (usando Canvas de HTML5) que dibuje la función en tiempo real. Añade controles deslizantes (sliders) para modificar los parámetros de cada función (como la pendiente 'm' o la curvatura 'a'), mostrando la ecuación resultante. En la columna lateral, el contenido debe actualizarse automáticamente mostrando la fórmula general, una breve explicación teórica y un 'Reto Rápido'.

Simulador de Compases y Ritmos

Herramienta interactiva para educación musical. Prompt: > Crea una aplicación web educativa e interactiva en un único archivo HTML titulada 'Simulador de Compases y Ritmos'. La interfaz debe tener una estética futurista, utilizando un tema oscuro con colores neón. Divídela en: un panel lateral izquierdo dedicado a la teoría musical y un panel principal a la derecha con un secuenciador de ritmos interactivo. > El secuenciador debe permitir crear ritmos para tres instrumentos: Bombo, Caja y Hi-Hat. Implementa controles para ajustar el Tempo (BPM) y botones para cambiar el tipo de compás (2/4, 3/4, 4/4 y 6/8). Incluye un motor de audio (Web Audio API) que genere los sonidos y un metrónomo visual que se sincronice con el tiempo fuerte de cada compás.

InvestLab: Simulador de Análisis Financiero

Análisis de salud corporativa para Economía. Prompt: > Crea una aplicación web interactiva en un único archivo HTML titulada 'InvestLab: Simulador de Análisis Financiero'. La interfaz debe ser elegante y corporativa, usando tonos azul marino y esmeralda. El núcleo es el panel de Análisis, que debe incluir una barra lateral para seleccionar entre diferentes tipos de empresas (Startup, Constructora, Retail). > Al seleccionar una empresa, actualiza dinámicamente indicadores como Resultado Neto, Activo Total y Ratio de Liquidez. Integra Chart.js para mostrar gráficos de barras y de tipo dónut para el Balance. Implementa ventanas modales que expliquen los conceptos financieros al hacer clic sobre ellos.

Juego Pasapalabra Educativo

Gamificación para el repaso de la unidad. Prompt: > Crea un juego de "Pasapalabra" educativo en un solo archivo HTML. > Requisitos técnicos: 1. Diseño responsivo con Tailwind CSS. 2. Un rosco central de la A a la Z que se ilumine en verde (acierto) o rojo (error). 3. Un panel lateral que muestre la definición actual. 4. Input para escribir la respuesta y botones de "Enviar" y "Pasapalabra". 5. Cronómetro de cuenta atrás. 6. Sonidos de éxito y error. 7. Pantalla de resultados final con resumen. Tema: [TEMA].

Multinivelación DUA (Inclusión)

Adaptación de actividades para la diversidad. Prompt: > Actúa como un experto en DUA (Diseño Universal para el Aprendizaje). Tengo la siguiente actividad: [DESCRIBIR ACTIVIDAD]. Genera una tabla comparativa con 3 niveles de adaptación: > 1. Nivel de Acceso: Inclusión con apoyos visuales, pictogramas y lectura fácil. > 2. Nivel Estándar: Basado en el currículo oficial. > 3. Nivel de Enriquecimiento: Mayor nivel de abstracción y creación para Altas Capacidades. Define para cada nivel el producto esperado, los criterios de evaluación y los recursos necesarios.

Guía de Publicación en Aules

Esta guía puede ser útil desde nivel básico si trabajas con archivo único HTML, y todavía más en nivel avanzado.

1. Copia el código de la IA y guárdalo en tu ordenador como un archivo con extensión `.html` (ej: `mi_recurso.html`).
2. Entra en tu curso de Aules y activa el Modo Edición.
3. Haz clic en "Añadir una actividad o recurso" y selecciona Archivo.
4. Sube el archivo HTML que has creado.
5. Configura la apariencia como "Incrustado" o "Abrir" según prefieras.

Consejos Finales de IA

- Si el código se corta: muchas veces la IA se detiene por límite de caracteres. Simplemente escribe: "Continúa exactamente donde lo dejaste".
- Pide bibliotecas externas solo cuando realmente te hagan falta: para gráficos puedes pedir Chart.js y para diseño Tailwind CSS. Recuerda pedir que las incluya vía CDN.
- Sé específico con el contexto: cuanta más información des sobre la asignatura y el nivel, mejor será el contenido educativo generado.

Créditos

Créditos — origen de los prompts

Las ideas sobre prompts de esta sección provienen de *Materiales y Prompts Estructurados para la Creación de Recursos Digitales con IA*, por **Ana Hernández Iranzo** (Empowered by AI).

Uso de la consola en navegadores (explicación amplia)

La consola del navegador es una herramienta de diagnóstico. Permite ver errores, advertencias y mensajes que ayudan a entender qué está pasando en tu página.

Cómo abrirla

- **Chrome / Edge / Brave (Windows/Linux):** `F12` o `Ctrl+Shift+I`
- **Chrome / Edge / Brave (Mac):** `Cmd+Option+I`
- **Firefox (Windows/Linux):** `F12` o `Ctrl+Shift+I`
- **Firefox (Mac):** `Cmd+Option+I`
- **Safari (Mac):** activa "Desarrollo" en Preferencias > Avanzado y luego `Cmd+Option+C`

Qué vas a ver

- **Errores (rojo):** fallos que impiden que el código funcione. Ejemplo: `Uncaught TypeError`.
- **Advertencias (amarillo):** algo puede fallar o ser mala práctica, pero no bloquea todo.
- **Mensajes (blanco):** lo que tú o las librerías imprimen con `console.log`.
- **Stack trace:** lista de archivos y líneas por donde pasó el error.

Flujo básico para depurar

1. Abre la consola.
2. Recarga la página para ver errores desde el inicio.
3. Reproduce el bug (clic, formulario, scroll).
4. Lee el primer error rojo de arriba hacia abajo.
5. Haz clic en el archivo y línea para abrir el código.

Mensajes comunes y qué significan

- `Uncaught TypeError: Cannot read properties of null`: estás intentando usar un elemento que no existe o no se encontró en el DOM.
- `Uncaught ReferenceError: x is not defined`: estás usando una variable que no fue declarada.
- `SyntaxError`: hay un error de sintaxis (llaves, paréntesis, comillas).
- `404 (Not Found)`: un archivo no se encontró (ruta incorrecta en HTML o JS).

Uso práctico de console.log

Usa `console.log()` para comprobar valores y flujo:

- Antes y después de una función para ver si se ejecuta.
- Para imprimir el valor de una variable o el resultado de una condición.
- Para confirmar que un evento se está disparando.

Ejemplo simple:

```
const boton = document.querySelector("#enviar");
console.log("Botón encontrado:", boton);
boton.addEventListener("click", () => {
  console.log("Click detectado");
```

```
});
```

Pistas rápidas para errores típicos

- Si la consola marca una línea en rojo, empieza por esa.
- Si sale `null`, revisa el `id` o `class` en tu HTML.
- Si el error es de ruta, revisa carpetas y nombres de archivo.

Copiar errores para pedir ayuda

1. Copia el mensaje completo del error.
2. Incluye el archivo y la línea.
3. Indica qué estabas haciendo cuando falló.

Cómo pedir a la IA un debug

- "En la consola veo: `Uncaught TypeError: ...` en `script.js:42`. Este es el código de esa función: … Quiero que me digas qué falla y un fix mínimo."
- "Mi botón no responde. En la consola aparece `Cannot read properties of null`. Estoy en `index.html` y `script.js`. Dime los pasos para localizar el bug."
- "Te paso el HTML y el JS. Crea un plan de depuración en 5 pasos y dime qué logs debo añadir."

Nota: pide siempre un cambio pequeño y comprobable antes de aplicar todo.

IA desde la terminal (CLI): Codex y Claude Code

> **Nivel avanzado.** Esta sección es opcional. Está pensada para quien ya se mueve con soltura con la IA en el chat y en VS Code, y quiere dar un paso más trabajando con un *agente* desde la terminal. Si todavía estás afianzando lo básico, puedes saltarla sin problema y volver más adelante.

Qué es una IA "CLI" y en qué se diferencia del chat

Hasta ahora has usado la IA de dos formas: en una web de chat (copias y pegas) o integrada en el editor (Copilot). Una **IA de línea de comandos (CLI)** es un tercer formato: un programa que se abre en la **terminal** y trabaja como un *agente* dentro de la carpeta de tu proyecto.

La diferencia clave es que **no solo te sugiere código: puede leer y modificar tus archivos, ejecutar comandos y comprobar resultados por sí mismo**, siempre pidiéndote permiso. Es como tener a la IA "sentada" dentro de tu proyecto en lugar de hablarle desde fuera.

Las dos herramientas más conocidas son:

- **Codex CLI** (de OpenAI): el agente de terminal asociado a ChatGPT.
- **Claude Code** (de Anthropic): el agente de terminal asociado a Claude. *Este mismo curso se ha elaborado con ayuda de estas herramientas.*

> Ambas evolucionan muy rápido. Los comandos y nombres exactos pueden cambiar; usa siempre la documentación oficial como referencia final.

Diferencia con Copilot: asistente frente a agente

Conviene aclarar en qué se diferencia esto del Copilot que ya conoces, porque se solapan bastante pero el modelo de trabajo es distinto.

- **El plugin de VS Code (Copilot) es un asistente.** Trabaja *alrededor de tu cursor*: te autocompleta mientras escribes y responde en el chat lateral. Tú abres cada archivo, aplicas cada cambio y ejecutas tú mismo los comandos.
- **La IA en CLI (Codex / Claude Code) es un agente.** Trabaja *sobre el proyecto entero*: decide qué archivos leer y tocar (aunque no los tengas abiertos) y puede **ejecutar comandos** —lanzar el proyecto, instalar algo, leer el error y reintentar— cerrando el ciclo por sí mismo, siempre pidiéndote permiso.

> **En una frase:** Copilot te ayuda a escribir el código **tú**; la CLI hace la tarea **ella** y tú la supervisas y apruebas.

Aspecto	Plugin VS Code (Copilot)	IA en CLI (Codex / Claude Code)
Rol	Asistente (sugiere)	Agente (ejecuta)
Ámbito	Lo que tienes abierto o seleccionado	Todo el proyecto (carpeta)
Ejecuta comandos	No (lo haces tú)	Sí (con permiso)
Ciclo "probar y corregir"	Manual	El agente lo puede cerrar solo
Varios archivos a la vez	Limitado	Es su punto fuerte
Curva de entrada	Baja	Requiere soltura con la terminal
Mejor para	Recursos de 1 archivo, escribir al momento	Proyectos con varios archivos, tareas repetitivas

Para la mayoría de recursos de este curso (un único HTML), el plugin sobra y tiene menos fricción. La CLI aporta de verdad en los casos que se describen a continuación.

Cuándo merece la pena (y cuándo no)

Merece la pena cuando:

- Trabajas con un proyecto de **varios archivos** y quieres que la IA tenga en cuenta el conjunto, no solo el fragmento que pegas.
- Quieres que la IA haga cambios y los **pruebe** ella misma (por ejemplo, instalar algo, lanzar el proyecto, leer el error y corregirlo).
- Repites tareas y te cansa copiar y pegar entre el chat y el editor.

No merece la pena (todavía) cuando:

- Estás haciendo un recurso sencillo de **un solo archivo HTML**: el chat o Copilot te sobran.
- No te sientes cómodo aún con la terminal: primero afianza Git y la terminal integrada (apunte *Herramientas, entorno y Git*).
- No quieres gestionar cuentas de pago o claves de API.

Requisito previo: Node.js

Las dos herramientas se instalan con **npm**, el gestor de paquetes que viene con **Node.js**. Comprueba si lo tienes abriendo la terminal integrada de VS Code (`Ver > Terminal`) y escribiendo:

```
node --version
npm --version
```

Si ves dos números de versión, ya lo tienes. Si no, instala Node.js desde su web oficial (`https://nodejs.org`) eligiendo la versión **LTS**, y vuelve a comprobarlo.

Instalar Codex CLI

```
npm install -g @openai/codex
```

Luego ábrelo desde la carpeta de tu proyecto:

```
codex
```

La primera vez te pedirá iniciar sesión. Puedes entrar con tu **cuenta de ChatGPT** (si tu plan lo incluye) o con una **clave de API** de OpenAI. Sigue las indicaciones que aparecen en pantalla.

Instalar Claude Code

```
npm install -g @anthropic-ai/claude-code
```

Luego ábrelo desde la carpeta de tu proyecto:

```
claude
```

La primera vez te pedirá autenticarte. Puedes entrar con tu **cuenta de Claude** (suscripción) o con una **clave de API** de Anthropic. Sigue las indicaciones que aparecen en pantalla.

> **Importante sobre la carpeta.** Ábrelas siempre *dentro* de la carpeta del proyecto en el que quieres trabajar (en VS Code, la terminal ya se abre ahí). El agente trabaja sobre los archivos de esa carpeta.

Flujo de trabajo básico

El uso, una vez dentro, es muy parecido a un chat, pero con superpoderes:

1. **Escribe una instrucción en lenguaje natural**, igual que harías en el chat. Por ejemplo: *"Crea un index.html con un cuestionario de 3 preguntas sobre fracciones para 1 ESO."*
2. **La IA te propone los cambios** (qué archivos va a crear o tocar) y **te pide permiso** antes de aplicarlos.
3. **Tú apruebas o rechazas** cada acción. Nunca modifica nada sin tu visto bueno.
4. **Puede ejecutar comandos** (lanzar el proyecto, instalar algo, leer errores) y corregir según el resultado, siempre pidiendo permiso.
5. **Sigues iterando** en la misma conversación: *"Ahora hazme los botones más grandes"*, *"Añade una cuarta pregunta"*, etc.

Para salir, normalmente se escribe `/exit` o se pulsa `Ctrl+C` un par de veces.

Ejemplo guiado (recurso educativo de principio a fin)

```
Tú: Quiero crear un recurso educativo en un único archivo HTML:
    un cuestionario de 3 preguntas sobre la fotosíntesis para 2 ESO,
    con botón de corregir y mensaje de feedback. Dame primero un plan
    de 3 etapas, no código todavía.

IA: [propone un plan por etapas]

Tú: Perfecto, haz solo la etapa 1.

IA: [crea index.html y te pide permiso para guardarlo]

Tú: [apruebas] Ahora ábrelo para comprobar que se ve bien.

IA: [lanza una vista previa / abre el archivo y confirma el resultado]
```

Fíjate en que el método es **el mismo que ya conoces** (planificar por etapas, cambios pequeños, comprobar antes de seguir): lo único que cambia es que la IA puede ejecutar los pasos por ti.

Buenas prácticas y avisos

- **Revisa siempre lo que propone antes de aprobar.** El agente puede crear, modificar o borrar archivos. Lee qué va a hacer; no apruebes "a ciegas".
- **Trabaja con Git.** Antes de una sesión larga, haz un `commit`. Así, si algo se tuerce, puedes volver atrás con facilidad (apunte *Herramientas, entorno y Git*).
- **Empieza con permisos restrictivos.** Aprueba acción por acción hasta que cojas confianza. Evita los modos "autónomos" o de "auto-aprobar todo" mientras estás aprendiendo.
- **Ten en cuenta el coste.** Según el plan, estas herramientas pueden consumir de tu suscripción o cobrar por uso de API. Revisa tu plan antes de lanzarte a sesiones largas.
- **Cuidado con los datos.** No incluyas en el proyecto datos personales del alumnado ni documentos sensibles. Vale lo mismo que para cualquier IA (consulta *Seguridad de datos* en el apunte de Buenas prácticas).
- **El criterio sigue siendo tuyo.** El agente es más rápido, pero no más responsable: la revisión pedagógica, la exactitud y la decisión final son tuyas.

Reglas persistentes (igual que en el chat)

Estas herramientas también leen un archivo de **instrucciones del proyecto** para mantener tus reglas entre sesiones (idioma, estilo, cambios pequeños y verificables…). El concepto es el mismo que viste en *Instrucciones y agentes*: escribe tus reglas una vez y reutilízalas. Consulta la documentación de cada herramienta para saber el nombre exacto del archivo (por convención suele ser un archivo de texto en la raíz del proyecto). # Primer proyecto guiado: Cuestionario Educativo

Antes de empezar, una aclaración importante: el cuestionario es el ejemplo principal porque es general,

fácil de entender y útil para la mayoría de aulas. Es un ejemplo de mínimos, pero puedes añadir todas las mejoras que se te ocurran para enriquecer y mejorar la aplicación. Si lo prefieres, puedes elegir otro tipo de proyecto siempre que sea práctico y útil para tu alumnado.

Descripción

Crea un cuestionario tipo test interactivo con HTML, CSS y un poco de JavaScript. El objetivo es crear una herramienta educativa simple, practicar el flujo de trabajo con Git cuando corresponda y publicar tu proyecto en Internet si quieres recorrer el itinerario avanzado.

Propuesta de proyecto

Un cuestionario online sobre un tema de tu elección que incluya:

- una o más preguntas de opción múltiple
- botones para que la persona usuaria seleccione una respuesta
- feedback visual para indicar si la respuesta es correcta o incorrecta

Proyectos alternativos equivalentes

Si el cuestionario no encaja con tu materia o tu estilo, puedes empezar con una plantilla alternativa del mismo nivel de dificultad:

> **Nota:** encontrarás estas plantillas en el repositorio oficial del curso: https://github.com/sasogu/vibecoding (carpeta `templates/`).

- `templates/ficha-vocabulario/`: para trabajar palabras clave, definiciones o conceptos.
- `templates/actividad-repaso/`: para preguntas breves con pista o ayuda.

Ruta por niveles

Nivel básico

Empieza con: - un `index.html` - una pregunta - varias respuestas visibles - un botón para “Corregir” - una comprobación en local en el navegador

Si quieres arrancar más rápido, puedes partir de una de estas plantillas:

- `templates/cuestionario-base/index.html`
- `templates/ficha-vocabulario/index.html`
- `templates/actividad-repaso/index.html`

Nivel medio

Añade: - `style.css` - `script.js` - estructura de carpetas - feedback más claro dentro de la página

Nivel avanzado

Añade: - publicación en GitHub Pages - varias preguntas - contador final - mejoras como PWA o adaptaciones para Aules/Moodle

Pasos guiados (Módulo 1 - Versión Mínima)

Nivel básico

1. Crea una carpeta para el proyecto.
   - Ejemplo de nombre de carpeta: `cuestionario-capitales`.
   - Abre la carpeta en VS Code con `Archivo > Abrir carpeta...`.
2. Opción recomendada: copia la carpeta `templates/cuestionario-base/` y renómbrala para tu proyecto.
3. Crea o edita el archivo `index.html` con una pregunta y cuatro posibles respuestas.
   - Las respuestas pueden ser texto simple o “radio buttons”, según tu nivel de comodidad.
4. Si empiezas desde cero, añade un botón para “Corregir”.
   - Si usas la plantilla, el botón ya viene preparado y puedes modificarlo.

5. Guarda y abre el archivo en el navegador.

Checklist de fin de etapa

- Se abre `index.html` en el navegador sin errores visibles.
- Se ve un título o texto inicial relacionado con tu recurso.
- Hay al menos una pregunta o contenido didáctico visible.
- Hay un botón o interacción básica preparada.

Nivel medio y avanzado

6. Inicia un repositorio de Git.
7. Sube tus cambios a un nuevo repositorio en GitHub.
   - Si no sabes usar la terminal, puedes hacer el `commit` y el `push` desde el panel de Git de VS Code.

   [IA] Cómo pedirle a la IA

   "Inicia el repositorio Git, haz el primer commit con todos los archivos del proyecto y súbelos al repositorio de GitHub que acabo de crear."

Capturas de apoyo (GitHub):

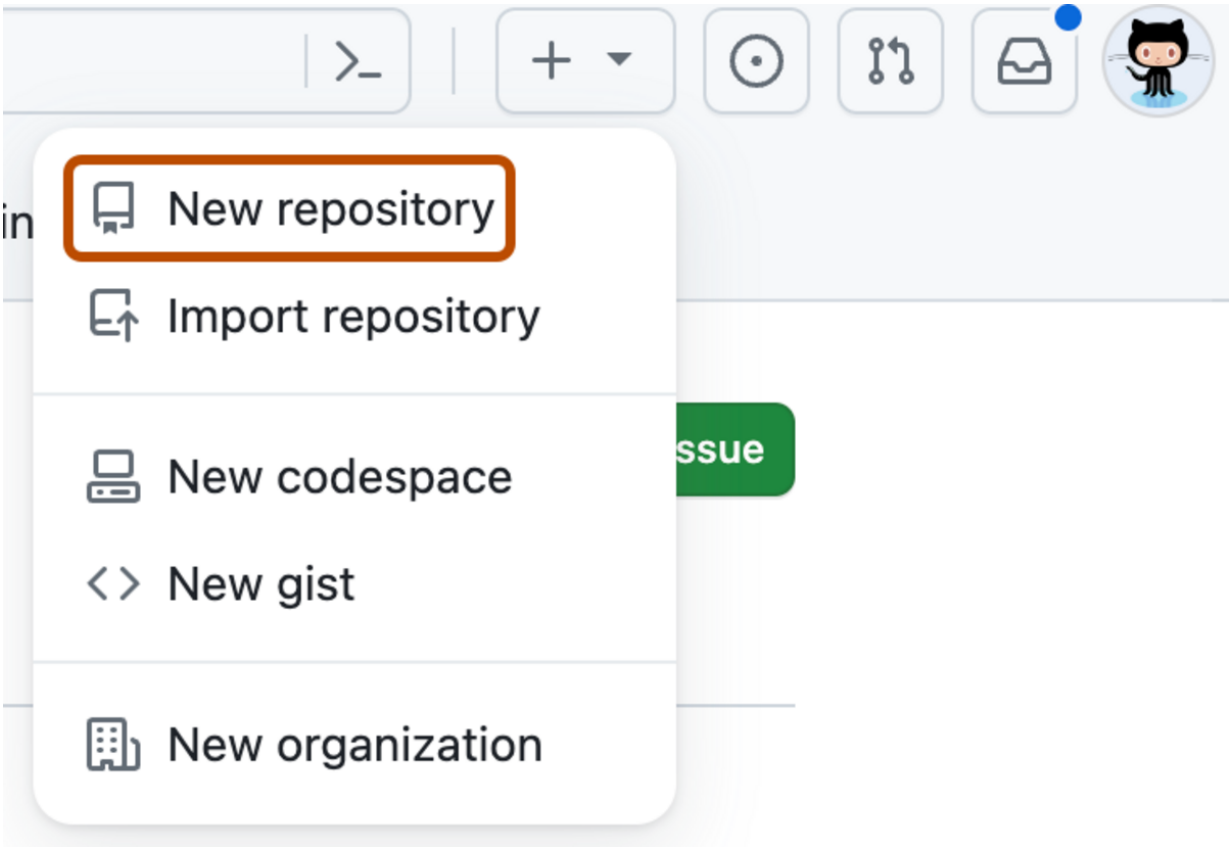


Figura 1: *Crear repositorio desde el menú*

Capturas tomadas de GitHub Docs (https://docs.github.com/).

Checklist de fin de etapa

- El proyecto mantiene una estructura clara y comprensible.
- Si estás en nivel medio, ya existe un `README.md` mínimo.
- Si estás en nivel avanzado, el repositorio remoto ya está creado.
- Sabes qué siguiente mejora vas a hacer antes de pedir más código a la IA.

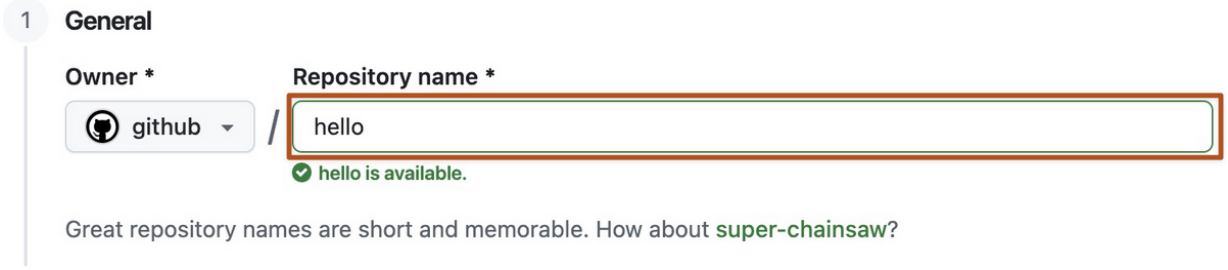


Figura 2: Nombre del repositorio

GitHub Pages: paso a paso

Esta parte forma parte sobre todo del nivel avanzado, aunque puedes probarla antes si ya te ves cómodo.

1. Entra al repositorio en GitHub.
2. Ve a **Settings > Pages**.
3. En **Build and deployment**, elige **Branch: `main` y / (root)**.
4. Guarda los cambios y espera a que aparezca la URL.
5. Abre la URL y verifica que carga `index.html`.

> **[IA] Cómo pedirle a la IA**
>
> "GitHub Pages no carga bien. Revisa la configuración del repositorio, identifica el problema y corrígelo."

Capturas de apoyo:

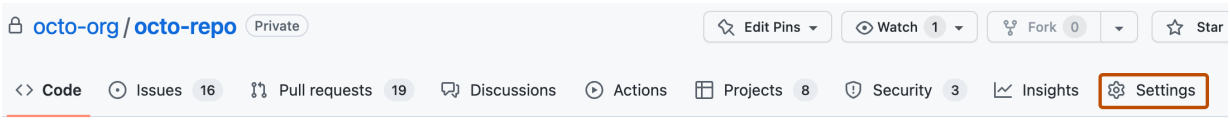


Figura 3: Pestaña Settings del repositorio

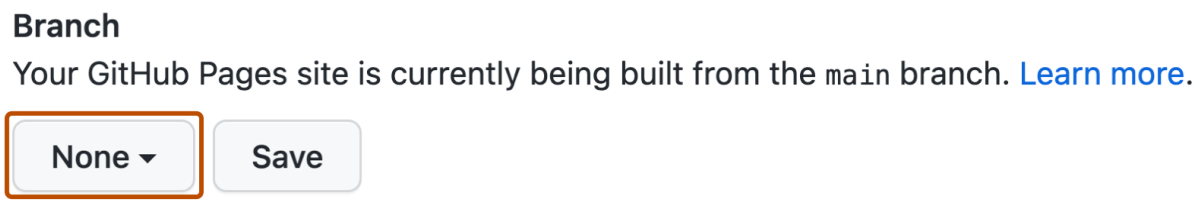


Figura 4: Selección de rama para publicar

Capturas tomadas de GitHub Docs (https://docs.github.com/).

(Opcional avanzado) ¿Qué es una PWA y qué ventajas ofrece en el aula?

Una **PWA** (Progressive Web App o "aplicación web progresiva") es una web que, además de abrirse en el navegador, puede comportarse como una app instalada.

Para conseguirlo suele incluir:

- un `manifest.json` (nombre, icono, pantalla de inicio)
- un *service worker* (caché y funcionamiento sin conexión parcial)
- servir por HTTPS (GitHub Pages ya lo hace)

Ventajas en dispositivos del aula

- **Acceso rápido:** icono en la pantalla de inicio, sin buscar la URL cada vez.
- **Experiencia tipo app:** pantalla completa, sin barra del navegador si se instala.
- **Mejor tolerancia a mala conexión:** puede cargar aunque el Wi-Fi falle (si el contenido está cachea-

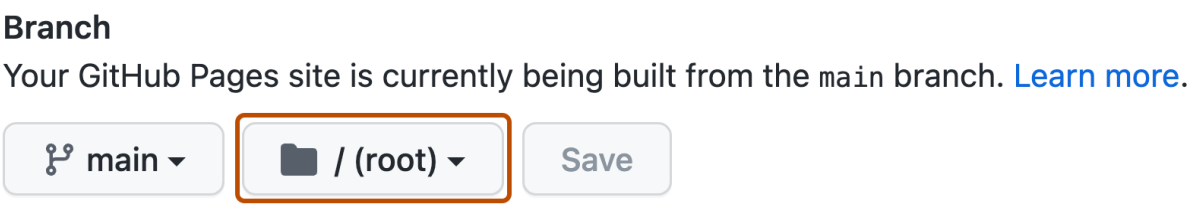


Figura 5: Selección de carpeta de publicación

do).
- **Multiplataforma:** funciona en Chromebooks, tablets y PCs sin pasar por una tienda.
- **Actualización sencilla:** al publicar cambios en GitHub Pages, el alumnado ve la versión actualizada.

Problemas comunes

- **No se ve la página en local:** revisa si has guardado el archivo.
- **Repositorio privado:** GitHub Pages no publica repos privados en el plan gratuito.
- **Branch incorrecta:** asegúrate de publicar desde `main` (o la rama que tengas como principal).
- **`index.html` fuera de raíz:** el archivo debe estar en la raíz del repositorio, no en una subcarpeta.

Entregable

Se aceptan varias formas de entrega

- archivo `index.html`
- carpeta del proyecto
- URL del proyecto publicado en GitHub Pages
- capturas o vídeo corto del recurso funcionando

Expectativa por niveles

- **Nivel básico:** `index.html` funcional o evidencia equivalente.
- **Nivel medio:** proyecto ordenado con `index.html`, archivos separados cuando proceda y `README.md` mínimo.
- **Nivel avanzado:** proyecto publicado o ampliado con funcionalidades extra.

Cómo usar este recurso en una sesión de clase

Una forma simple de llevar el recurso al aula es pensar en una secuencia corta:

1. **Inicio:** presentar el tema y explicar qué va a hacer el alumnado con el recurso.
2. **Uso guiado:** resolver una primera pregunta o ejemplo en grupo para asegurarse de que todo el mundo entiende la dinámica.
3. **Trabajo individual o por parejas:** dejar que el alumnado complete el recurso, pruebe respuestas o repase contenidos.
4. **Cierre:** comentar errores frecuentes, recoger dudas y decidir qué mejorarías para la siguiente versión.

Variantes por materia

- **Lenguas:** vocabulario, comprensión lectora, ortografía, repaso gramatical.
- **Ciencias:** clasificación, conceptos clave, procesos, autoevaluación rápida.
- **Geografía e Historia:** fechas, mapas, causas y consecuencias, repaso de unidades.
- **Matemáticas:** cálculo mental, problemas cortos, identificación de conceptos.
- **FP:** procedimientos, secuencias de trabajo, prevención de errores frecuentes.

Guía mínima de uso

Si acompañas el proyecto con una guía breve, intenta responder a estas preguntas:

- para qué curso o grupo está pensado
- qué necesita saber el alumnado antes de usarlo

- cuánto tiempo ocupa
- qué aprendizaje o repaso pretende reforzar
- qué mejora harías después de probarlo

Módulo 2: Añadir CSS y JavaScript

El objetivo de este módulo es que el cuestionario tenga un aspecto más cuidado (CSS) y empiece a funcionar (JavaScript).

- **Imprescindible:** aplicar estilos básicos y comprobar funcionamiento.
- **Opcional:** separar CSS y JavaScript en archivos externos si te da tiempo.
- **Tiempo orientativo:** 35-50 min (básico) + 25-35 min por ampliación.

Nivel básico

Ejercicio 1: añadir estilos dentro del HTML **Qué vas a hacer:** mejorar el aspecto de la página sin crear archivos externos. Puedes añadir los estilos directamente dentro del `<head>` del HTML.

Pasos:

1. Abre `index.html` en VS Code.
2. Busca la etiqueta `<head>` y añade un bloque `<style>` dentro.
3. Pídele a la IA que añada estilos básicos:

Prompt sugerido:

```
Añade estilos CSS dentro del <head> de este index.html para:
- Cambiar la fuente a Arial o una fuente sans-serif
- Añadir un margen de 20px alrededor del contenido
- Poner el fondo de la página en color claro (por ejemplo #f5f5f5)
- Hacer que el botón "Corregir" tenga un color destacado y buen tamaño
No crees archivos externos todavía.
```

4. Guarda y recarga el navegador para ver los cambios.

Resultado esperado: la página tiene mejor aspecto, con márgenes, una fuente legible y un botón con color.

Checklist de fin de etapa

☐ La página tiene estilos visibles (no es solo texto negro sobre fondo blanco).
☐ El botón destaca del resto del contenido.
☐ No hay errores visibles en la consola del navegador (`F12 > Console`).

Nivel medio

Ejercicio 2: separar el CSS en un archivo externo **Qué vas a hacer:** mover los estilos a un archivo `style.css` para tener el proyecto mejor organizado.

Pasos:

1. Crea una carpeta `css` dentro de tu proyecto.
2. Dentro de esa carpeta, crea el archivo `style.css`.
3. Pídele a la IA que mueva los estilos:

Prompt sugerido:

```
Mueve todos los estilos que hay dentro del <style> de index.html
al archivo css/style.css y enlaza ese archivo desde el <head> de index.html.
Asegúrate de que la página sigue viéndose igual después del cambio.
```

4. Guarda ambos archivos y comprueba que la página sigue viéndose igual en el navegador.

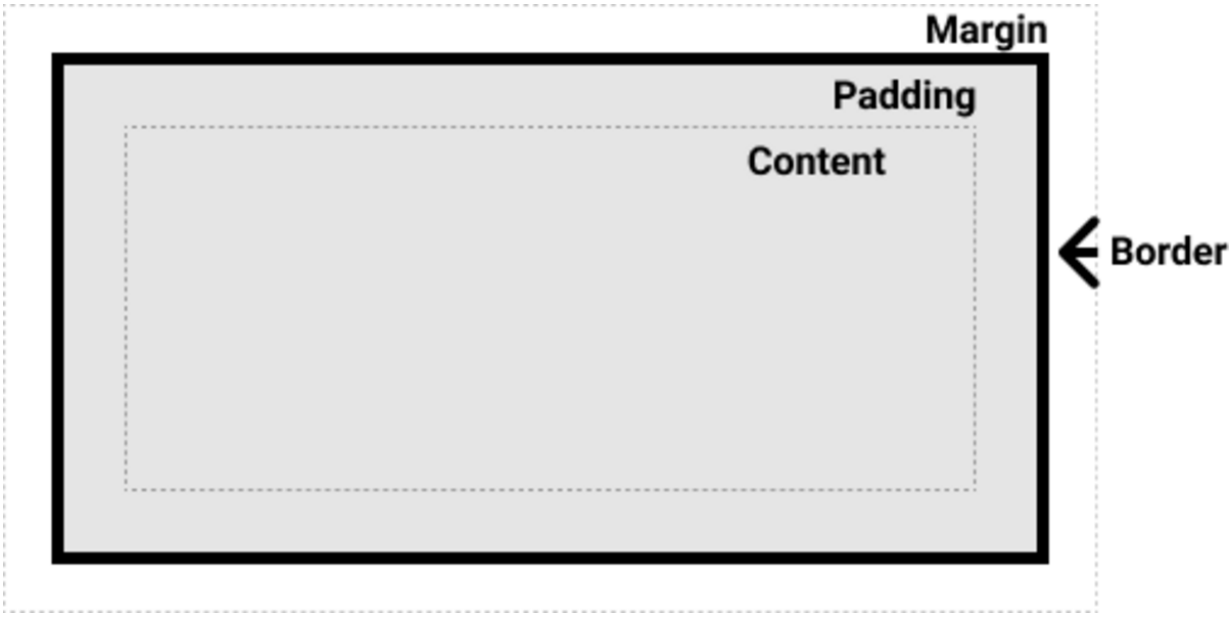


Figura 6: Modelo de caja CSS

Cómo se enlaza el CSS (para tu referencia):

```
<head>
  <link rel="stylesheet" href="css/style.css">
</head>
```

Ejercicio 3: separar el JavaScript en un archivo externo Qué vas a hacer: crear un archivo `script.js` con una función que compruebe si la respuesta es correcta.

Pasos:

1. Crea una carpeta `js` dentro de tu proyecto.
2. Dentro de esa carpeta, crea el archivo `script.js`.
3. Pídele a la IA que cree la función de corrección:

Prompt sugerido:

```
Crea una función JavaScript en js/script.js que:
- Se ejecute al pulsar el botón "Corregir"
- Compruebe qué radio button está seleccionado
- Muestre un alert() que diga "Correcto" si la respuesta elegida es la correcta,
  o "Incorrecto" si no lo es
La respuesta correcta es la opción [indica aquí cuál es, por ejemplo: "Madrid"].
Enlaza también el script desde index.html justo antes de </body>.
```

4. Comprueba en el navegador que el botón ahora muestra un mensaje al pulsarlo.

Cómo se enlaza el JavaScript (para tu referencia):

```
  <script src="js/script.js"></script>
</body>
```

Checklist de fin de etapa

☐ Existe el archivo `css/style.css` y está enlazado desde `index.html`.
☐ Existe el archivo `js/script.js` y está enlazado desde `index.html`.
☐ Al pulsar "Corregir" aparece un mensaje indicando si la respuesta es correcta.
☐ No hay errores en la consola del navegador (`F12 > Console`).

Nivel avanzado

Reto: añadir al menos dos preguntas y gestionar ambas con el script Prompt sugerido:

```
El cuestionario ahora tiene solo una pregunta. Añade una segunda pregunta con
sus respuestas al index.html y actualiza script.js para que el botón "Corregir"
compruebe las dos preguntas a la vez y muestre cuántas ha acertado el usuario.
```

Módulo 2: Añadir CSS y JavaScript

Si quieres ir más allá de la tarea del módulo (dar estilo con CSS y añadir una primera interacción con JavaScript), aquí tienes algunos recursos seleccionados para profundizar por tu cuenta.

Documentación para consultar

- MDN — Primeros pasos con CSS: tutorial oficial en español; cubre tipografía, márgenes y colores, justo lo que pide la tarea del módulo.
- MDN — Primeros pasos con JavaScript: explica los eventos de botón y el uso de la consola del navegador, la herramienta clave para comprobar que no hay errores.

Vídeo recomendado

- Curso completo de introducción al desarrollo web (HTML, CSS y JavaScript) — gratis: repaso completo y gratuito que enlaza con el HTML visto en el módulo 1 y avanza a CSS y JavaScript.

Para seguir explorando

- Google Fonts: tipografías gratuitas para mejorar el aspecto visual de tu recurso.